MODUL 13 · PROYEK BAGIAN 1

Proyek Catatan Kuliah (1): Arsitektur & Autentikasi Supabase

Memulai proyek utuh 'Catatan Kuliah': menyusun kebutuhan, arsitektur MVC berlapis (View, Controller GetX, Service/Repository, Supabase/Hive), struktur folder, tabel profiles dengan trigger, lalu membangun fitur register, login, logout, dan sesi tersimpan menggunakan Supabase Auth, GetX (AuthController), dan AuthGate.

Durasi Pertemuan 13 (150 Menit) + Belajar Mandiri

Prasyarat Modul 05: Form & Validasi Input
Modul 07: Navigasi
Modul 08: State Management GetX & MVC
Modul 12: Supabase

Capaian Pembelajaran

- Menyusun kebutuhan fungsional dan alur layar sebuah aplikasi mobile sebelum menulis kode.
- Menerapkan arsitektur MVC berlapis View, Controller (GetX), Service/Repository dengan dependency injection Get.put/Get.find.
- Membuat tabel profiles beserta trigger handle_new_user dan policy RLS di Supabase.
- Mengimplementasikan AuthService dan AuthController (state reaktif .obs, onInit/onClose) dengan status loading dan pesan error berbahasa Indonesia.
- Membangun AuthGate berbasis Obx dan onAuthStateChange untuk navigasi login/logout otomatis.
- Membuat LoginView dan RegisterView dengan validasi form, show/hide password, tombol loading reaktif, dan SnackBar.
- Menguji alur autentikasi secara sistematis, termasuk mode konfirmasi email dan sesi tersimpan.

Daftar Isi Modul

1. Gambaran Proyek & Kebutuhan
2. Arsitektur, Struktur Folder & Package
3. Setup Supabase: Tabel profiles & Trigger
4. Env, Validator & AuthService
5. AuthController & Pesan Error Ramah
6. AuthGate, Home Sementara & main.dart
7. LoginView
8. RegisterView & Menjalankan Aplikasi
9. Pengujian & Troubleshooting
10. Tugas Praktikum

11. Kuis Pemahaman & Kunci Jawaban

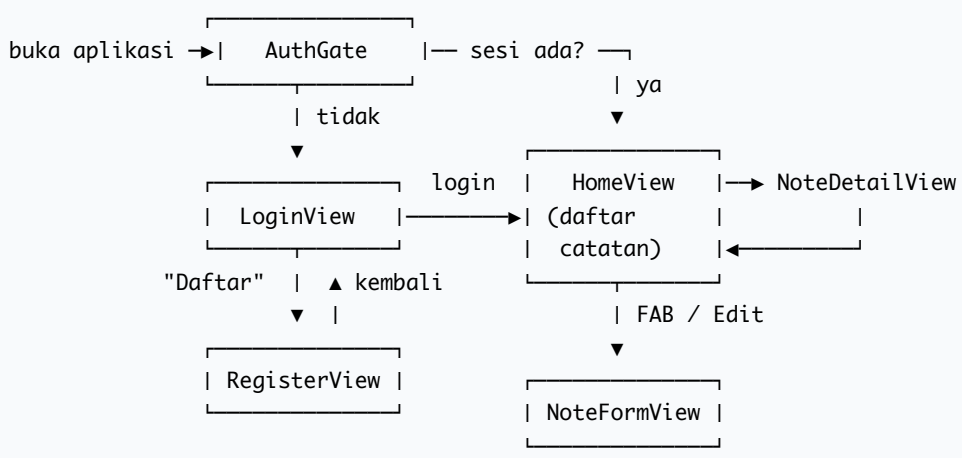
1. Gambaran Proyek & Kebutuhan

Di Modul 13 dan 14 kita membangun satu aplikasi utuh dari proyek kosong sampai siap dipakai: **Catatan Kuliah**. Mahasiswa bisa mendaftar, login, lalu menulis catatan per mata kuliah yang tersimpan di cloud (Supabase) dan tetap bisa dibaca saat offline (cache Hive).

Kebutuhan (requirements)

Kode	Kebutuhan	Dikerjakan di
F-01	User bisa **register** dengan nama lengkap, email, dan password.	Modul 13
F-02	User bisa **login** dan **logout**. Sesi tetap tersimpan saat aplikasi ditutup.	Modul 13
F-03	Pesan error dalam Bahasa Indonesia (password salah, email belum dikonfirmasi, dan lain-lain).	Modul 13
F-04	User bisa **menambah, melihat, mengubah, dan menghapus** catatan (CRUD).	Modul 14
F-05	User **hanya** bisa melihat catatannya sendiri (RLS).	Modul 14
F-06	Pencarian judul/isi dan filter kategori (Kuliah, Tugas, Ujian, Lainnya).	Modul 14
F-07	Saat offline, catatan terakhir tetap tampil dari cache dengan banner "mode offline".	Modul 14
F-08	Logout menghapus cache lokal agar data tidak terbaca user berikutnya.	Modul 14

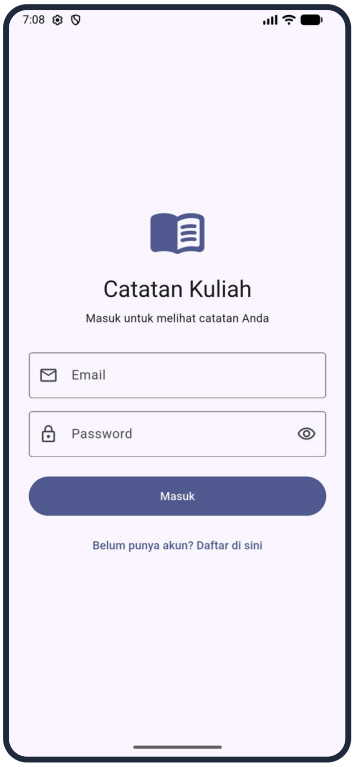
Alur layar



Modul 13 menyelesaikan bagian kiri (AuthGate, Login, Register) beserta `HomeView` sementara. Modul 14 mengganti `HomeView` dengan daftar catatan lengkap, lalu menambahkan form dan detail.

Hasil akhir yang ingin dicapai

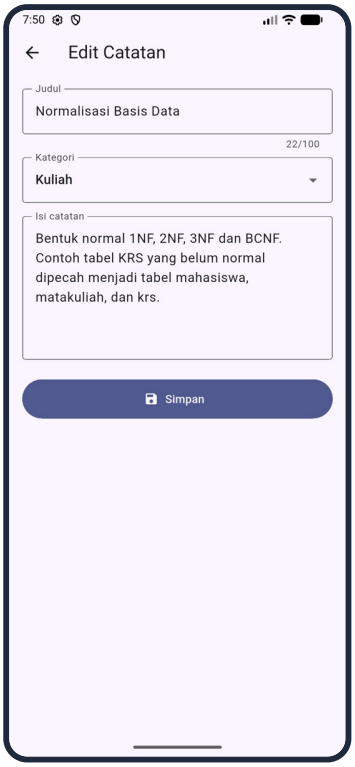
Tangkapan layar aplikasi final (akhir Modul 14) yang terhubung ke project Supabase:



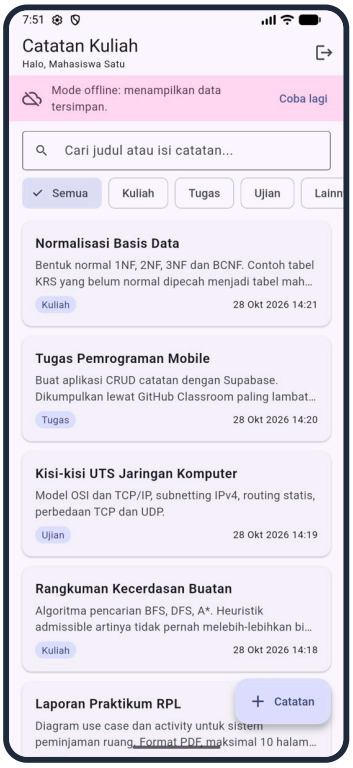
Login



Daftar catatan milik user



Form tambah/edit



Mode offline dari cache Hive

Apa yang akan Anda latih

- **Form & validasi** (Modul 05): TextFormField, GlobalKey<FormState>, validator.
- **Navigasi** (Modul 07): push/pop, ditambah navigasi otomatis berbasis status login.
- **GetX & MVC** (Modul 08): `GetxController`, state reaktif `.obs`, `Obx`, `Get.put` / `Get.find`, `onInit` / `onClose`, navigasi `Get.to` / `Get.back`.
- **JSON & repository** (Modul 09): model fromJson/toJson, pemisahan lapisan data.
- **Hive** (Modul 11): box sebagai cache offline.
- **Supabase** (Modul 12): Auth, SQL, RLS, query CRUD.

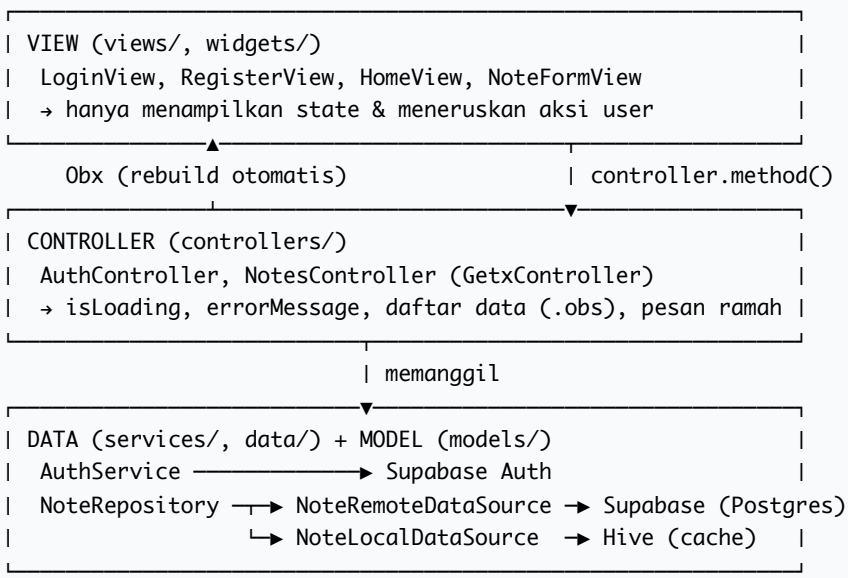
> 💡 **Cara mengikuti proyek ini**
>
> Kerjakan **berurutan**. Setiap section menambah satu atau beberapa file, dan di akhir section yang membuat aplikasi bisa dijalankan, ada langkah "jalankan & cek". Jika ada yang error, bandingkan file Anda dengan listing lengkap di akhir Modul 14.

2. Arsitektur, Struktur Folder & Package

Sebelum menulis kode, kita sepakati dulu "denah rumah"-nya. Arsitektur yang rapi membuat kode mudah diuji, mudah dicari saat error, dan mudah dikembangkan.

Lapisan (layer) aplikasi: pola MVC dengan GetX

Proyek ini memakai pola **MVC** yang sudah dikenalkan di Modul 08: **View** menampilkan, **Controller** mengelola state dan logika, **Model** + lapisan data mengurus penyimpanan. Lapisan data dipecah lagi menjadi *service* (Auth) dan *repository* (catatan).



Lapisan	Boleh tahu tentang	Tidak boleh tahu tentang
View	Controller, model, widget	Supabase, Hive, SQL
Controller	Service/Repository, model, jenis exception, `package:get`	Widget, `BuildContext`, SnackBar, navigasi
Service/Repository	Supabase client, Hive box, model	Controller, View, GetX

Aturan sederhananya: **panah hanya ke bawah** (View → Controller → Service → Model). Karena `NoteRepository` tidak mengenal View maupun GetX, kita bisa mengujinya dengan unit test tanpa emulator (dilakukan di Modul 14). Controller pun tidak menampilkan SnackBar atau berpindah halaman sendiri; ia cukup mengembalikan hasil (`bool`, enum, atau pesan error), lalu View yang memutuskan reaksinya.

ℹ️ Bagaimana View mendapatkan controller?

Lewat *dependency injection* GetX. Di `main()` kita mendaftarkan controller sekali dengan `Get.put(AuthController(AuthService(...)), permanent: true)`. Halaman mana pun lalu mengambilnya dengan `Get.find<AuthController>()` atau dengan `extends GetView<AuthController>`. Service disuntikkan lewat constructor controller, sehingga saat testing kita bisa memberi service palsu.

Struktur folder akhir

```
catatan_kuliah/
├── env.json                  ← URL & publishable key (TIDAK di-commit)
├── env.example.json          ← contoh isi env.json (di-commit)
├── supabase/
│   └── setup.sql             ← semua SQL proyek (disimpan untuk dokumentasi)
├── lib/
│   ├── main.dart             ← inisialisasi Hive + Supabase, Get.put
│   ├── config/
│   │   └── env.dart          ← membaca --dart-define             [M13]
│   ├── models/
│   │   └── note.dart         ← model Note + fromJson/toJson/toMap [M14]
│   ├── services/
│   │   └── auth_service.dart ← signUp/signIn/signOut             [M13]
│   ├── data/
│   │   ├── note_remote_data_source.dart  ← CRUD Supabase         [M14]
│   │   ├── note_local_data_source.dart   ← cache Hive            [M14]
│   │   └── note_repository.dart          ← gabungan + offline    [M14]
│   ├── controllers/
│   │   ├── auth_controller.dart          ← state login           [M13]
│   │   └── notes_controller.dart         ← state catatan         [M14]
│   ├── views/
│   │   ├── auth_gate.dart                ← Login atau Home?      [M13]
│   │   ├── login_view.dart                                       [M13]
│   │   ├── register_view.dart                                    [M13]
│   │   ├── home_view.dart                ← sementara di M13, final di M14
│   │   ├── note_form_view.dart           ← tambah & edit         [M14]
│   │   └── note_detail_view.dart                                 [M14]
│   ├── widgets/
│   │   ├── confirm_dialog.dart                                   [M14]
│   │   ├── note_card.dart                                        [M14]
│   │   └── offline_banner.dart                                   [M14]
│   └── utils/
│       ├── validators.dart               ← validator form        [M13]
│       └── date_format.dart                                      [M14]
└── test/
    ├── note_local_data_source_test.dart                          [M14]
    └── note_repository_test.dart                                 [M14]
```

Langkah 1: Membuat proyek dan memasang package

```
flutter create catatan_kuliah
cd catatan_kuliah

flutter pub add supabase_flutter get hive_ce hive_ce_flutter
```

Package	Kegunaan di proyek
`supabase_flutter`	Auth dan database cloud (Modul 12)
`get`	GetX: state management reaktif, dependency injection, dan navigasi (Modul 08)
`hive_ce`	Database key-value lokal untuk cache (Modul 11)
`hive_ce_flutter`	`Hive.initFlutter()`: menentukan folder penyimpanan di HP

Setelah perintah di atas, bagian `dependencies` di `pubspec.yaml` kira-kira seperti ini (nomor versi mengikuti yang terbaru saat Anda menjalankan perintah):

`pubspec.yaml`

```yaml
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  supabase_flutter: ^x.y.z
  get: ^x.y.z
  hive_ce: ^x.y.z
  hive_ce_flutter: ^x.y.z
```

Langkah 2: Siapkan folder dan file konfigurasi

```
mkdir -p lib/config lib/models lib/services lib/data lib/controllers lib/views lib/widgets lib/utils
```

Buat `env.json` di root proyek (sejajar `pubspec.yaml`) dan isi dengan nilai dari dashboard Supabase Anda (Modul 12 section 2):

`env.json`

```json
{
  "SUPABASE_URL": "https://YOUR-PROJECT-REF.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_YOUR_KEY"
}
```

Tambahkan ke `.gitignore`:

`.gitignore`

```
# Kunci Supabase
env.json
```

Lalu salin `env.json` menjadi `env.example.json` dengan nilai placeholder, agar teman satu tim tahu format yang dibutuhkan.

Terakhir, tambahkan izin internet untuk build release di `android/app/src/main/AndroidManifest.xml` (di atas tag `<application>`):

```
<uses-permission android:name="android.permission.INTERNET"/>
```

3. Setup Supabase: Tabel profiles & Trigger

Aplikasi kita butuh tempat menyimpan data tambahan tentang user, misalnya nama lengkap. Tabel `auth.users` dikelola Supabase dan tidak bisa diakses langsung dari aplikasi, sehingga pola yang lazim adalah membuat tabel `public.profiles` yang barisnya dibuat **otomatis** oleh trigger setiap kali ada user baru.

Alur data saat register

```
Flutter: auth.signUp(email, password, data: {'full_name': 'Fikri Ramadhan'})
   |
   ▼
auth.users  ← baris baru, raw_user_meta_data = {"full_name": "Fikri Ramadhan"}
   |  (trigger on_auth_user_created: AFTER INSERT)
   ▼
public.profiles ← (id = id user baru, full_name = 'Fikri Ramadhan')
```

Langkah 1: Jalankan SQL berikut di SQL Editor

Buka **SQL Editor → New query**, tempel seluruh SQL di bawah, lalu klik **Run**. SQL ini aman dijalankan ulang karena memakai `if not exists`, `drop ... if exists`, dan `create or replace`.

`supabase/setup.sql`

```sql
create table if not exists public.profiles (
  id uuid primary key references auth.users (id) on delete cascade,
  full_name text,
  created_at timestamptz not null default now()
);

alter table public.profiles enable row level security;

drop policy if exists "User melihat profilnya sendiri" on public.profiles;
create policy "User melihat profilnya sendiri"
on public.profiles for select
to authenticated
using ((select auth.uid()) = id);

drop policy if exists "User mengubah profilnya sendiri" on public.profiles;
create policy "User mengubah profilnya sendiri"
on public.profiles for update
to authenticated
using ((select auth.uid()) = id)
with check ((select auth.uid()) = id);

create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into public.profiles (id, full_name)
  values (new.id, new.raw_user_meta_data ->> 'full_name');
```

```
  return new;
end;
$$;

drop trigger if exists on_auth_user_created on auth.users;
create trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_user();
```

Penjelasan per bagian

Kode	Penjelasan
`id uuid primary key references auth.users (id)`	id profil **sama** dengan id user (relasi satu-satu). `on delete cascade` : profil ikut terhapus jika akun dihapus.
`enable row level security`	Mengunci tabel. Tanpa policy, tidak ada yang bisa membaca.
Policy select/update dengan `(select auth.uid()) = id`	User hanya bisa membaca dan mengubah profilnya sendiri. Tidak ada policy INSERT karena insert dilakukan oleh trigger, bukan oleh aplikasi.
`returns trigger`	Fungsi khusus yang dipanggil oleh trigger. Variabel `new` berisi baris yang baru dimasukkan ke `auth.users`.
`security definer`	Fungsi berjalan dengan hak *pembuat* fungsi (admin), bukan hak user. Ini perlu karena pada saat register user belum login, dan RLS profiles akan menolaknya.
`set search_path = ''`	Praktik keamanan untuk fungsi `security definer` : semua nama tabel harus ditulis lengkap (`public.profiles`) sehingga tidak bisa "dibelokkan" ke tabel lain.
`new.raw_user_meta_data ->> 'full_name'`	Operator `->>` mengambil nilai dari kolom JSON sebagai teks. Isinya berasal dari parameter `data` di `signUp()`.
`after insert on auth.users for each row`	Trigger dijalankan sekali untuk setiap user baru.

> ⚠️ **Trigger error = register gagal**
>
> Jika fungsi trigger error (misalnya salah ketik nama tabel), **seluruh** proses register ikut dibatalkan dan aplikasi menerima pesan *Database error saving new user*. Jika itu terjadi, buka **Logs → Postgres** di dashboard untuk melihat detail error-nya.

Langkah 2: Atur konfirmasi email

1. Buka **Authentication → Sign In / Providers → Email**.
2. Untuk praktikum di kelas, **matikan** *Confirm email* agar register langsung login dan tidak terkena batas pengiriman email.
3. Aplikasi yang kita buat tetap mendukung kedua mode: jika Confirm email aktif, RegisterView menampilkan dialog "Cek email Anda".

Langkah 3: Verifikasi

Setelah aplikasi selesai (section 8), register satu akun lalu buka **Table Editor → profiles**. Harus ada satu baris dengan `full_name` sesuai yang Anda isi. Akun yang sama juga terlihat di **Authentication → Users**.

4. Env, Validator & AuthService

Kita mulai dari lapisan paling bawah, lalu naik ke atas: konfigurasi → service → controller → view. Di section ini kita membuat tiga file yang tidak bergantung pada widget sama sekali.

File 1: `lib/config/env.dart`

Membaca URL dan key yang disuntikkan lewat `--dart-define-from-file=env.json` (Modul 12 section 4).

`lib/config/env.dart`

```dart
/// Konfigurasi Supabase yang dibaca saat build lewat --dart-define.
///
/// Jalankan aplikasi dengan:
///   flutter run --dart-define-from-file=env.json
class Env {
  static const String supabaseUrl = String.fromEnvironment('SUPABASE_URL');
  static const String supabasePublishableKey = String.fromEnvironment(
    'SUPABASE_PUBLISHABLE_KEY',
  );

  /// true jika kedua nilai sudah diisi.
  static bool get isConfigured =>
      supabaseUrl.isNotEmpty && supabasePublishableKey.isNotEmpty;
}
```

File 2: `lib/utils/validators.dart`

Validator dipakai oleh form Login dan Register. Kita kumpulkan di satu tempat agar aturannya konsisten (misalnya minimal 6 karakter mengikuti aturan default Supabase).

`lib/utils/validators.dart`

```dart
/// Kumpulan validator untuk TextFormField.
/// Setiap fungsi mengembalikan null jika valid, atau pesan error jika tidak.
class Validators {
  static String? fullName(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Nama lengkap wajib diisi';
    if (text.length < 3) return 'Nama minimal 3 karakter';
    return null;
  }

  static String? email(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Email wajib diisi';
    final pola = RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$');
    if (!pola.hasMatch(text)) return 'Format email tidak valid';
    return null;
  }

  static String? password(String? value) {
    final text = value ?? '';
    if (text.isEmpty) return 'Password wajib diisi';
```

```
    if (text.length < 6) return 'Password minimal 6 karakter';
    return null;
  }
}
```

- Setiap validator mengembalikan `null` jika valid, atau `String` pesan error jika tidak valid. Ini kontrak `validator` pada `TextFormField` (Modul 05).
- Method dibuat `static` sehingga bisa langsung dipasang sebagai *tear-off*: `validator: Validators.email`.
- Regex email sengaja dibuat sederhana. Validasi sesungguhnya tetap dilakukan Supabase (error `email_address_invalid`).

File 3: `lib/services/auth_service.dart`

`AuthService` adalah **satu-satunya** class yang memanggil `supabase.auth`. Class lain cukup memakai service ini.

`lib/services/auth_service.dart`

```dart
import 'package:supabase_flutter/supabase_flutter.dart';

/// Lapisan paling bawah untuk urusan autentikasi.
/// Hanya class ini yang berbicara langsung dengan Supabase Auth.
class AuthService {
  AuthService(this._client);

  final SupabaseClient _client;

  GoTrueClient get _auth => _client.auth;

  /// Stream yang memancarkan event setiap status login berubah
  /// (initialSession, signedIn, signedOut, tokenRefreshed, ...).
  Stream<AuthState> get authStateChanges => _auth.onAuthStateChange;

  Session? get currentSession => _auth.currentSession;

  User? get currentUser => _auth.currentUser;

  /// Mendaftarkan akun baru. Nama lengkap dikirim sebagai metadata
  /// (raw_user_meta_data) dan nanti disalin trigger ke tabel profiles.
  Future<AuthResponse> signUp({
    required String fullName,
    required String email,
    required String password,
  }) {
    return _auth.signUp(
      email: email,
      password: password,
      data: {'full_name': fullName},
    );
  }

  Future<AuthResponse> signIn({
    required String email,
    required String password,
```

```
  }) {
    return _auth.signInWithPassword(email: email, password: password);
  }

  Future<void> signOut() => _auth.signOut();
}
```

Penjelasan:

- **Constructor menerima `SupabaseClient`** (*dependency injection*), bukan memanggil `Supabase.instance.client` di dalam class. Dengan begitu kita bisa mengganti client saat testing.
- `authStateChanges` meneruskan stream `onAuthStateChange`. Stream ini yang nanti didengarkan oleh `AuthController` agar AuthGate tahu kapan harus berganti halaman.
- `signUp(... data: {'full_name': fullName})`: nama lengkap dikirim sebagai metadata, lalu trigger `handle_new_user` (section 3) menyalinnya ke tabel `profiles`.
- Method mengembalikan `Future<AuthResponse>` apa adanya. Penanganan error *bukan* tugas service, melainkan tugas controller di section berikutnya.

> ℹ️ **Kenapa tidak langsung memanggil Supabase dari widget?**
>
> Bisa saja, dan untuk demo kecil itu wajar. Tetapi jika suatu hari Anda pindah ke Firebase atau backend kampus sendiri, Anda hanya perlu mengganti isi `AuthService`. Controller dan semua halaman (view) tidak perlu disentuh.

5. AuthController & Pesan Error Ramah

`AuthController` adalah huruf **C** dalam MVC: ia menjembatani View (halaman) dan `AuthService`. Tugasnya: menyimpan status *loading* dan status login secara reaktif, menangkap exception, lalu menerjemahkannya menjadi pesan Bahasa Indonesia yang ramah pengguna.

Pasang dulu package GetX bila belum (sudah termasuk di perintah section 2):

```
flutter pub add get
```

File 4: `lib/controllers/auth_controller.dart`

`lib/controllers/auth_controller.dart`

```dart
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:get/get.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../services/auth_service.dart';

/// Hasil proses registrasi, dipakai RegisterView untuk menentukan
/// pesan yang ditampilkan.
enum RegisterResult { failed, loggedIn, needsEmailConfirmation }

class AuthController extends GetxController {
  AuthController(this._authService);

  final AuthService _authService;

  // State privat yang reaktif.
  final _isLoading = false.obs;
  final _errorMessage = RxnString();
  final _session = Rxn<Session>();

  StreamSubscription<AuthState>? _authSubscription;

  // Getter publik read-only. Membaca getter ini di dalam Obx
  // otomatis mendaftarkan Obx sebagai pendengar.
  bool get isLoading => _isLoading.value;
  String? get errorMessage => _errorMessage.value;

  Stream<AuthState> get authStateChanges => _authService.authStateChanges;
  Session? get currentSession => _session.value;
  User? get currentUser => _session.value?.user;

  /// Nama yang ditampilkan di AppBar. Diambil dari metadata saat register.
  String get displayName {
    final user = currentUser;
    if (user == null) return '';
    final name = user.userMetadata?['full_name'];
    if (name is String && name.isNotEmpty) return name;
```

```
    return user.email ?? '';
  }

  @override
  void onInit() {
    super.onInit();
    // Mulai dari sesi yang tersimpan di perangkat (jika ada) ...
    _session.value = _authService.currentSession;
    // ... lalu ikuti setiap perubahan status login.
    _authSubscription = _authService.authStateChanges.listen(
      (data) => _session.value = data.session,
      onError: (Object e) => debugPrint('authStateChanges: $e'),
    );
  }

  @override
  void onClose() {
    _authSubscription?.cancel();
    super.onClose();
  }

  Future<bool> signIn({required String email, required String password}) async {
    _setLoading(true);
    try {
      await _authService.signIn(email: email.trim(), password: password);
      return true;
    } on AuthException catch (e) {
      _errorMessage.value = _mapAuthError(e);
      return false;
    } catch (e) {
      _errorMessage.value = 'Terjadi kesalahan tak terduga: $e';
      return false;
    } finally {
      _setLoading(false);
    }
  }

  Future<RegisterResult> register({
    required String fullName,
    required String email,
    required String password,
  }) async {
    _setLoading(true);
    try {
      final response = await _authService.signUp(
        fullName: fullName.trim(),
        email: email.trim(),
        password: password,
      );
      // Jika "Confirm email" aktif di dashboard, session bernilai null
      // sampai pengguna mengeklik link di email.
      if (response.session == null) {
        return RegisterResult.needsEmailConfirmation;
      }
      return RegisterResult.loggedIn;
    } on AuthException catch (e) {
      _errorMessage.value = _mapAuthError(e);
      return RegisterResult.failed;
    } catch (e) {
```

```
      _errorMessage.value = 'Terjadi kesalahan tak terduga: $e';
      return RegisterResult.failed;
    } finally {
      _setLoading(false);
    }
  }

  Future<void> signOut() async {
    try {
      await _authService.signOut();
    } on AuthException catch (e) {
      // signOut tetap menghapus sesi lokal walau request ke server gagal.
      debugPrint('signOut: ${e.message}');
    }
  }

  void clearError() {
    _errorMessage.value = null;
  }

  void _setLoading(bool value) {
    _isLoading.value = value;
    if (value) _errorMessage.value = null;
  }

  /// Menerjemahkan error Supabase Auth menjadi pesan berbahasa Indonesia.
  String _mapAuthError(AuthException e) {
    if (e is AuthRetryableFetchException) {
      return 'Tidak dapat terhubung ke server. Periksa koneksi internet.';
    }
    switch (e.code) {
      case 'invalid_credentials':
        return 'Email atau password salah.';
      case 'email_not_confirmed':
        return 'Email belum dikonfirmasi. Buka link verifikasi di inbox Anda.';
      case 'user_already_exists':
      case 'email_exists':
        return 'Email sudah terdaftar. Silakan login.';
      case 'weak_password':
        return 'Password terlalu lemah. Gunakan minimal 6 karakter.';
      case 'email_address_invalid':
        return 'Alamat email tidak valid.';
      case 'over_email_send_rate_limit':
      case 'over_request_rate_limit':
        return 'Terlalu banyak percobaan. Tunggu beberapa menit lalu coba lagi.';
      case 'signup_disabled':
        return 'Pendaftaran akun baru sedang dinonaktifkan.';
    }
    return e.message;
  }
}
```

Membedah AuthController

Bagian	Penjelasan
`enum RegisterResult`	Register punya **tiga** kemungkinan hasil: gagal, langsung login, atau perlu konfirmasi email. Enum membuat View bisa memakai `switch` yang jelas, alih-alih `bool` yang ambigu.
`_isLoading = false.obs`, `_errorMessage = RxnString()`	State privat yang reaktif (Modul 08). `RxnString` adalah `Rx` untuk `String?` yang boleh bernilai `null`. View hanya membaca lewat getter publik `isLoading` dan `errorMessage`, sehingga tidak bisa mengubah state dari luar.
`_session = Rxn<Session>()`	Salinan reaktif status login. `currentSession` dan `currentUser` membaca nilai ini, jadi widget di dalam `Obx` otomatis dibangun ulang saat user login atau logout.
`onInit()`	Dipanggil sekali saat controller didaftarkan dengan `Get.put`. Di sini kita mengisi sesi awal dari perangkat, lalu `listen` ke `authStateChanges` (stream `onAuthStateChange`). Setiap event membawa `session` terbaru.
`onClose()`	Pasangan `onInit`: membatalkan `StreamSubscription` agar tidak bocor memori bila controller dihapus.
`displayName`	Getter turunan: mengambil `full_name` dari `userMetadata`, dengan email sebagai cadangan jika nama kosong. Karena membaca `currentUser`, getter ini juga reaktif di dalam `Obx`.
`try / on AuthException / catch / finally`	`finally` **selalu** dijalankan, baik sukses maupun gagal, sehingga loading pasti dimatikan. Tanpa `finally`, spinner bisa berputar selamanya jika terjadi error.
`response.session == null`	Tanda bahwa Confirm email aktif: akun dibuat tetapi belum login.
`_setLoading(true)`	Sekaligus menghapus error lama, agar pesan error percobaan sebelumnya tidak muncul lagi. Tidak ada `update()` atau pemberitahuan manual: mengubah `.value` sudah cukup.
`signOut()`	Error saat logout hanya dicatat dengan `debugPrint`. `signOut` tetap menghapus sesi lokal walaupun request ke server gagal (misalnya offline), jadi user tetap keluar.

ℹ️ **Controller tidak mengenal widget**

Perhatikan import di atas: tidak ada `material.dart`, tidak ada `BuildContext`, tidak ada SnackBar atau navigasi. Controller hanya **mengembalikan hasil** (`bool` atau `RegisterResult`) dan menyimpan `errorMessage`. View-lah yang memutuskan cara menampilkannya. GetX sebenarnya menyediakan `Get.snackbar` yang bisa dipanggil dari controller, tetapi di kursus ini kita sengaja tidak melakukannya agar controller tetap mudah di-unit-test tanpa widget.

Pemetaan error ke pesan ramah

`AuthException` memiliki properti `code` berisi kode error dari server Supabase Auth. Kode-kode yang kita tangani:

`e.code`	Kapan muncul	Pesan untuk user
`invalid_credentials`	Email tidak terdaftar atau password salah	Email atau password salah.
`email_not_confirmed`	Login sebelum klik link konfirmasi	Email belum dikonfirmasi...
`user_already_exists` / `email_exists`	Register dengan email yang sudah dipakai (Confirm email OFF)	Email sudah terdaftar. Silakan login.
`weak_password`	Password kurang dari batas minimal	Password terlalu lemah...
`over_email_send_rate_limit`	Terlalu banyak email konfirmasi terkirim	Terlalu banyak percobaan...
(AuthRetryableFetchException)	Tidak ada internet / server tidak terjangkau	Tidak dapat terhubung ke server...

💡 **Kenapa "Email atau password salah", bukan "Email tidak terdaftar"?**

Supabase sengaja tidak membedakan kedua kasus. Pesan yang terlalu spesifik membantu penyerang menebak email mana yang terdaftar. Pesan gabungan seperti ini adalah praktik keamanan standar.

6. AuthGate, Home Sementara & main.dart

Bagaimana aplikasi tahu harus menampilkan Login atau Home? Kita **tidak** memakai `Get.off` atau `Navigator.pushReplacement` setelah login. Sebagai gantinya, satu widget bernama `AuthGate` mengamati status login di `AuthController` (yang mendengarkan stream `onAuthStateChange`) dan memilih halaman secara otomatis.

Kenapa AuthGate lebih baik daripada navigasi manual?

Situasi	Navigasi manual	AuthGate (reaktif)
Login berhasil	Harus memanggil `Get.off(...)` / `pushReplacement` ke Home	Otomatis pindah ke Home
Logout	Harus `Get.offAll(...)` / `pushAndRemoveUntil` ke Login	Otomatis kembali ke Login
Aplikasi dibuka ulang, sesi masih ada	Perlu cek manual di splash screen	Langsung ke Home (event `initialSession`)
Sesi kedaluwarsa / dicabut	Sering terlupa	Otomatis ke Login (event `signedOut`)

File 5: `lib/views/auth_gate.dart`

`lib/views/auth_gate.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/auth_controller.dart';
import 'home_view.dart';
import 'login_view.dart';

/// "Penjaga pintu" aplikasi: mengamati status login di AuthController
/// lalu memilih halaman yang tepat secara otomatis.
class AuthGate extends GetView<AuthController> {
  const AuthGate({super.key});

  @override
  Widget build(BuildContext context) {
    return Obx(() {
      // currentSession dibaca di dalam Obx, jadi setiap kali sesi berubah
      // (signedIn, signedOut, ...) builder ini dijalankan ulang.
      if (controller.currentSession == null) {
        return const LoginView();
      }
      return const HomeView();
    });
```

```
  }
}
```

- `extends GetView<AuthController>` : `GetView` adalah `StatelessWidget` yang punya getter `controller`, isinya sama dengan `Get.find<AuthController>()`. Cocok untuk halaman yang hanya memakai satu controller.
- `Obx(() { ... })` membaca `controller.currentSession`. Karena getter itu membaca `_session.value`, Obx otomatis mendaftar sebagai pendengar dan dibangun ulang setiap kali sesi berubah.
- Tidak perlu `StatefulWidget`, `initState`, atau `StreamBuilder`. Langganan stream sudah diurus `AuthController` di `onInit` dan dibatalkan di `onClose`, sehingga tidak ada risiko berlangganan ulang di setiap build.
- Sesi awal diisi dari `currentSession` di `onInit`, sehingga user yang sudah login tidak melihat layar Login "berkedip" sesaat ketika aplikasi dibuka.
- Setiap event (`signedIn`, `signedOut`, `tokenRefreshed`, ...) membawa `session` terbaru, jadi cukup cek `currentSession == null`.

File 6 (sementara): `lib/views/home_view.dart`

Untuk menguji alur login, kita buat Home sederhana yang menampilkan nama dan email user serta tombol keluar. File ini akan **diganti seluruhnya** di Modul 14.

`lib/views/home_view.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/auth_controller.dart';

/// HomeView sementara untuk Bagian 1. Di Modul 14 file ini diganti
/// dengan daftar catatan lengkap.
class HomeView extends GetView<AuthController> {
  const HomeView({super.key});

  Future<void> _logout() async {
    await controller.signOut();
    // Tidak perlu navigasi: AuthGate otomatis menampilkan LoginView.
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Catatan Kuliah'),
        actions: [
          IconButton(
            tooltip: 'Keluar',
            icon: const Icon(Icons.logout),
            onPressed: _logout,
          ),
        ],
```

```
      ),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Obx(() {
            final user = controller.currentUser;
            return Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                const Icon(Icons.verified_user, size: 72, color: Colors.green),
                const SizedBox(height: 16),
                Text(
                  'Halo, ${controller.displayName}!',
                  style: Theme.of(context).textTheme.headlineSmall,
                  textAlign: TextAlign.center,
                ),
                const SizedBox(height: 8),
                Text('Login sebagai ${user?.email ?? '-'}'),
                const SizedBox(height: 4),
                SelectableText(
                  'User ID: ${user?.id ?? '-'}',
                  style: Theme.of(context).textTheme.bodySmall,
                  textAlign: TextAlign.center,
                ),
                const SizedBox(height: 24),
                const Text(
                  'Autentikasi berhasil. Daftar catatan dibuat di Modul 14.',
                  textAlign: TextAlign.center,
                ),
                const SizedBox(height: 24),
                OutlinedButton.icon(
                  onPressed: _logout,
                  icon: const Icon(Icons.logout),
                  label: const Text('Keluar'),
                ),
              ],
            );
          }),
        ),
      ),
    );
  }
}
```

- `extends GetView<AuthController>` menyediakan getter `controller` (hasil `Get.find`), sama seperti di AuthGate.
- Hanya bagian yang membaca data user yang dibungkus `Obx`. `AppBar` tidak bergantung pada state, jadi tidak perlu ikut dibangun ulang.
- Perhatikan `_logout`: setelah `signOut()` tidak ada navigasi sama sekali (tidak ada `Get.offAll` atau `Navigator`). Stream memancarkan `signedOut`, `AuthController` mengosongkan sesi, lalu `Obx` di AuthGate menampilkan LoginView.

File 7: `lib/main.dart` (versi Bagian 1)

`lib/main.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'config/env.dart';
import 'controllers/auth_controller.dart';
import 'services/auth_service.dart';
import 'views/auth_gate.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  if (!Env.isConfigured) {
    runApp(const ConfigErrorApp());
    return;
  }

  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabasePublishableKey,
  );

  // Daftarkan controller global SEBELUM runApp. permanent: true membuat
  // AuthController tidak pernah dihapus selama aplikasi berjalan.
  Get.put(
    AuthController(AuthService(Supabase.instance.client)),
    permanent: true,
  );

  runApp(const CatatanKuliahApp());
}

class CatatanKuliahApp extends StatelessWidget {
  const CatatanKuliahApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      title: 'Catatan Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
      ),
      home: const AuthGate(),
    );
  }
}

/// Ditampilkan jika aplikasi dijalankan tanpa --dart-define-from-file.
class ConfigErrorApp extends StatelessWidget {
  const ConfigErrorApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: Center(
          child: Padding(
            padding: EdgeInsets.all(24),
            child: Text(
              'SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY belum diisi.\n\n'
              'Jalankan dengan:\n'
              'flutter run --dart-define-from-file=env.json',
              textAlign: TextAlign.center,
            ),
          ),
        ),
      ),
    );
  }
}
```

- `if (!Env.isConfigured)`: jika lupa `--dart-define-from-file`, aplikasi menampilkan petunjuk yang jelas, bukan error membingungkan dari Supabase.
- `AuthService(Supabase.instance.client)` dibuat *sesudah* `initialize` selesai, lalu disuntikkan ke constructor `AuthController`.
- **`Get.put(..., permanent: true)` sebelum `runApp`** mendaftarkan `AuthController` sebagai controller global. Saat itu juga `onInit` dijalankan sehingga langganan status login sudah aktif sebelum layar pertama tampil. `permanent: true` mencegah GetX menghapus controller ini ketika halaman ditutup. Di Modul 14, `NotesController` didaftarkan di tempat yang sama.
- `GetMaterialApp` menggantikan `MaterialApp` agar fitur GetX seperti `Get.to` dan `Get.back` (dipakai di LoginView dan RegisterView) bisa berjalan. Parameter lain (`title`, `theme`, `home`) tetap sama.
- `ConfigErrorApp` cukup memakai `MaterialApp` biasa karena tidak memakai controller maupun navigasi GetX.

> ⚠️ **Belum bisa dijalankan**
>
> `auth_gate.dart` meng-import `login_view.dart`, yang baru kita buat di section berikutnya. Wajar jika editor masih menandai error merah. Lanjutkan ke section 7 dan 8.

Jika aplikasi dijalankan tanpa `--dart-define-from-file`, tampilannya seperti ini:



ConfigErrorApp: muncul saat env.json tidak diberikan

7. LoginView

LoginView menggabungkan materi Form (Modul 05) dan GetX (Modul 08): validasi input, tombol show/hide password, tombol dengan indikator loading yang reaktif, dan SnackBar saat gagal.

File 8: `lib/views/login_view.dart`

`lib/views/login_view.dart`

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/auth_controller.dart';
import '../utils/validators.dart';
import 'register_view.dart';

class LoginView extends StatefulWidget {
  const LoginView({super.key});

  @override
  State<LoginView> createState() => _LoginViewState();
}

class _LoginViewState extends State<LoginView> {
  final _auth = Get.find<AuthController>();
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  bool _obscurePassword = true;

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    // Tutup keyboard lalu validasi form.
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    final success = await _auth.signIn(
      email: _emailController.text,
      password: _passwordController.text,
    );

    // Jika sukses, AuthGate otomatis pindah ke HomeView.
    if (!success && mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(_auth.errorMessage ?? 'Login gagal'),
          backgroundColor: Theme.of(context).colorScheme.error,
        ),
      );
```

```
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: SafeArea(
      child: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                Icon(
                  Icons.menu_book_rounded,
                  size: 72,
                  color: Theme.of(context).colorScheme.primary,
                ),
                const SizedBox(height: 16),
                Text(
                  'Catatan Kuliah',
                  textAlign: TextAlign.center,
                  style: Theme.of(context).textTheme.headlineMedium,
                ),
                const SizedBox(height: 8),
                const Text(
                  'Masuk untuk melihat catatan Anda',
                  textAlign: TextAlign.center,
                ),
                const SizedBox(height: 32),
                TextFormField(
                  controller: _emailController,
                  keyboardType: TextInputType.emailAddress,
                  textInputAction: TextInputAction.next,
                  autofillHints: const [AutofillHints.email],
                  decoration: const InputDecoration(
                    labelText: 'Email',
                    prefixIcon: Icon(Icons.email_outlined),
                    border: OutlineInputBorder(),
                  ),
                  validator: Validators.email,
                ),
                const SizedBox(height: 16),
                TextFormField(
                  controller: _passwordController,
                  obscureText: _obscurePassword,
                  textInputAction: TextInputAction.done,
                  onFieldSubmitted: (_) => _submit(),
                  decoration: InputDecoration(
                    labelText: 'Password',
                    prefixIcon: const Icon(Icons.lock_outline),
                    border: const OutlineInputBorder(),
                    suffixIcon: IconButton(
                      tooltip: _obscurePassword
                          ? 'Tampilkan password'
                          : 'Sembunyikan password',
                      icon: Icon(
```

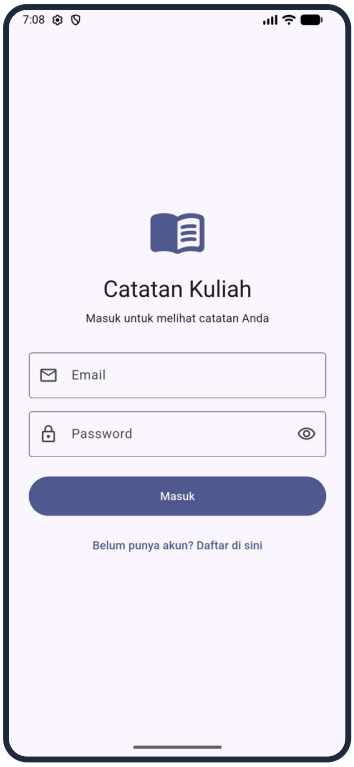
```
                        _obscurePassword
                            ? Icons.visibility_outlined
                            : Icons.visibility_off_outlined,
                      ),
                      onPressed: () {
                        setState(() => _obscurePassword = !_obscurePassword);
                      },
                    ),
                  ),
                  validator: Validators.password,
                ),
                const SizedBox(height: 24),
                // Hanya bagian tombol yang bergantung pada isLoading,
                // jadi hanya bagian ini yang dibungkus Obx.
                Obx(() {
                  final isLoading = _auth.isLoading;
                  return Column(
                    crossAxisAlignment: CrossAxisAlignment.stretch,
                    children: [
                      FilledButton(
                        onPressed: isLoading ? null : _submit,
                        style: FilledButton.styleFrom(
                          minimumSize: const Size.fromHeight(48),
                        ),
                        child: isLoading
                            ? const SizedBox(
                                width: 22,
                                height: 22,
                                child: CircularProgressIndicator(
                                  strokeWidth: 2.5,
                                ),
                              )
                            : const Text('Masuk'),
                      ),
                      const SizedBox(height: 12),
                      TextButton(
                        onPressed: isLoading
                            ? null
                            : () => Get.to(() => const RegisterView()),
                        child: const Text('Belum punya akun? Daftar di sini'),
                      ),
                    ],
                  );
                }),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

Membedah LoginView

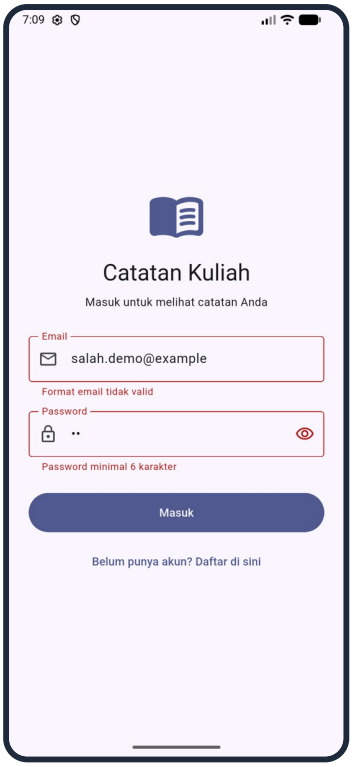
Bagian	Penjelasan
`StatefulWidget`	Butuh state lokal: dua controller, `GlobalKey<FormState>`, dan `_obscurePassword`. Ini *ephemeral state* milik satu halaman, jadi cukup `setState`. State yang dipakai bersama (loading, sesi, pesan error) tetap ada di `AuthController`.
`dispose()`	Controller wajib di-dispose agar tidak bocor memori.
`FocusScope.of(context).unfocus()`	Menutup keyboard sebelum proses login, supaya SnackBar tidak tertutup keyboard.
`_formKey.currentState!.validate()`	Menjalankan semua validator. Jika ada yang gagal, pesan merah muncul di bawah field dan proses berhenti.
`final _auth = Get.find<AuthController>()`	Mengambil controller yang sudah didaftarkan dengan `Get.put` di `main()`. Karena LoginView adalah `StatefulWidget` (tidak bisa memakai `GetView`), controller disimpan sebagai field di State dan dipakai di `_submit` maupun `build`.
`if (!success && mounted)`	Setelah `await`, widget mungkin sudah tidak ada. Saat login sukses, LoginView justru **dibuang** oleh AuthGate, sehingga pengecekan `mounted` wajib sebelum memakai `context`.
`Obx(() { final isLoading = _auth.isLoading; ... })`	Hanya kedua tombol yang bergantung pada `isLoading`, jadi hanya bagian itu yang dibungkus `Obx`. Saat loading berubah, form dan field tidak ikut dibangun ulang.
`onPressed: isLoading ? null : _submit`	`onPressed: null` membuat tombol nonaktif, sehingga mencegah klik ganda yang mengirim dua request login.
`suffixIcon: IconButton(...)`	Tombol mata untuk menampilkan atau menyembunyikan password dengan membalik nilai `obscureText`.
`onFieldSubmitted: (_) => _submit()`	Menekan tombol "selesai" di keyboard langsung memicu login.

Bagian	Penjelasan
`Get.to(() => const RegisterView())`	Register dibuka di atas Login (setara `Navigator.push`), dan tombol back mengembalikan user ke Login. Tidak perlu meneruskan controller ke halaman baru: RegisterView cukup memanggil `Get.find`.

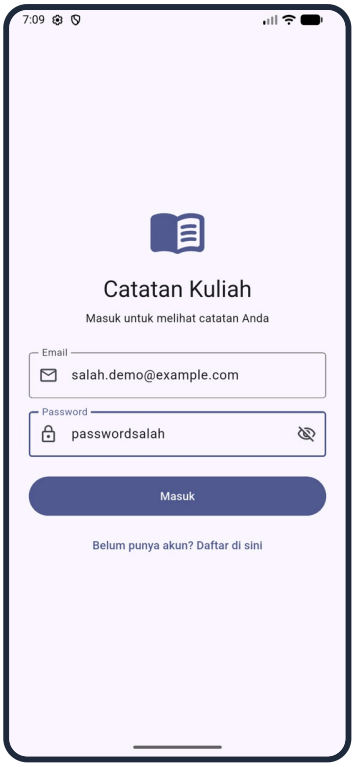
Hasil di emulator dengan kode di atas:



Tampilan awal LoginView



Validasi: format email salah dan password kurang dari 6 karakter



Ikon mata ditekan: password terlihat



Password salah: AuthException dari Supabase diterjemahkan menjadi pesan ramah

⚠️ **Kesalahan umum**

- **Membungkus `Obx` di sekitar widget yang tidak membaca nilai Rx apa pun**: muncul error *"[Get] the improper use of a GetX has been detected"*. Pastikan `_auth.isLoading` dibaca *di dalam* fungsi builder `Obx`, bukan di luar lalu dikirim sebagai variabel.
- **Memanggil `Get.off(() => const HomeView())` setelah login**: Home akan muncul dua kali (satu dari navigasi, satu dari AuthGate), dan logout jadi kacau.
- **Lupa cek `mounted`**: muncul warning `use_build_context_synchronously` dan berpotensi crash.
- **Memanggil `Get.find<AuthController>()` sebelum `Get.put`**: muncul error *"AuthController not found"*. Pastikan `Get.put` di `main()` dijalankan sebelum `runApp`.

8. RegisterView & Menjalankan Aplikasi

RegisterView mirip LoginView, ditambah field nama lengkap, konfirmasi password, dan penanganan tiga kemungkinan hasil register.

File 9: `lib/views/register_view.dart`

`lib/views/register_view.dart`

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/auth_controller.dart';
import '../utils/validators.dart';

class RegisterView extends StatefulWidget {
  const RegisterView({super.key});

  @override
  State<RegisterView> createState() => _RegisterViewState();
}

class _RegisterViewState extends State<RegisterView> {
  final _auth = Get.find<AuthController>();
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _confirmController = TextEditingController();
  bool _obscurePassword = true;

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _confirmController.dispose();
    super.dispose();
  }

  String? _validateConfirm(String? value) {
    if (value == null || value.isEmpty) return 'Ulangi password Anda';
    if (value != _passwordController.text) return 'Password tidak sama';
    return null;
  }

  Future<void> _submit() async {
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    final result = await _auth.register(
      fullName: _nameController.text,
      email: _emailController.text,
      password: _passwordController.text,
    );
    if (!mounted) return;
```

```
    switch (result) {
      case RegisterResult.loggedIn:
        // Sesi langsung aktif. AuthGate di bawah halaman ini sudah berganti
        // ke HomeView, jadi cukup tutup halaman register.
        Get.back();
      case RegisterResult.needsEmailConfirmation:
        await showDialog<void>(
          context: context,
          builder: (dialogContext) => AlertDialog(
            title: const Text('Cek Email Anda'),
            content: Text(
              'Link konfirmasi sudah dikirim ke ${_emailController.text.trim()}. '
              'Klik link tersebut, lalu login dengan akun baru Anda.',
            ),
            actions: [
              FilledButton(
                onPressed: () => Navigator.of(dialogContext).pop(),
                child: const Text('Mengerti'),
              ),
            ],
          ),
        );
        if (mounted) Get.back();
      case RegisterResult.failed:
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text(_auth.errorMessage ?? 'Registrasi gagal'),
            backgroundColor: Theme.of(context).colorScheme.error,
          ),
        );
    }
  }

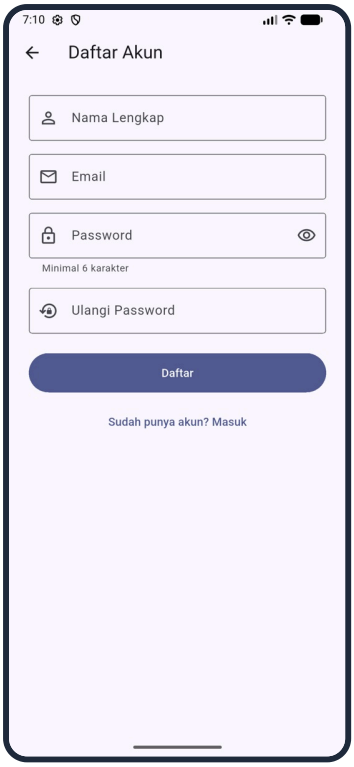
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Daftar Akun')),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                TextFormField(
                  controller: _nameController,
                  textCapitalization: TextCapitalization.words,
                  textInputAction: TextInputAction.next,
                  decoration: const InputDecoration(
                    labelText: 'Nama Lengkap',
                    prefixIcon: Icon(Icons.person_outline),
                    border: OutlineInputBorder(),
                  ),
                  validator: Validators.fullName,
                ),
                const SizedBox(height: 16),
                TextFormField(
```

```
      controller: _emailController,
      keyboardType: TextInputType.emailAddress,
      textInputAction: TextInputAction.next,
      decoration: const InputDecoration(
        labelText: 'Email',
        prefixIcon: Icon(Icons.email_outlined),
        border: OutlineInputBorder(),
      ),
      validator: Validators.email,
    ),
    const SizedBox(height: 16),
    TextFormField(
      controller: _passwordController,
      obscureText: _obscurePassword,
      textInputAction: TextInputAction.next,
      decoration: InputDecoration(
        labelText: 'Password',
        helperText: 'Minimal 6 karakter',
        prefixIcon: const Icon(Icons.lock_outline),
        border: const OutlineInputBorder(),
        suffixIcon: IconButton(
          icon: Icon(
            _obscurePassword
                ? Icons.visibility_outlined
                : Icons.visibility_off_outlined,
          ),
          onPressed: () {
            setState(() => _obscurePassword = !_obscurePassword);
          },
        ),
      ),
      validator: Validators.password,
    ),
    const SizedBox(height: 16),
    TextFormField(
      controller: _confirmController,
      obscureText: _obscurePassword,
      textInputAction: TextInputAction.done,
      onFieldSubmitted: (_) => _submit(),
      decoration: const InputDecoration(
        labelText: 'Ulangi Password',
        prefixIcon: Icon(Icons.lock_reset_outlined),
        border: OutlineInputBorder(),
      ),
      validator: _validateConfirm,
    ),
    const SizedBox(height: 24),
    Obx(() {
      final isLoading = _auth.isLoading;
      return Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          FilledButton(
            onPressed: isLoading ? null : _submit,
            style: FilledButton.styleFrom(
              minimumSize: const Size.fromHeight(48),
            ),
            child: isLoading
                ? const SizedBox(
```

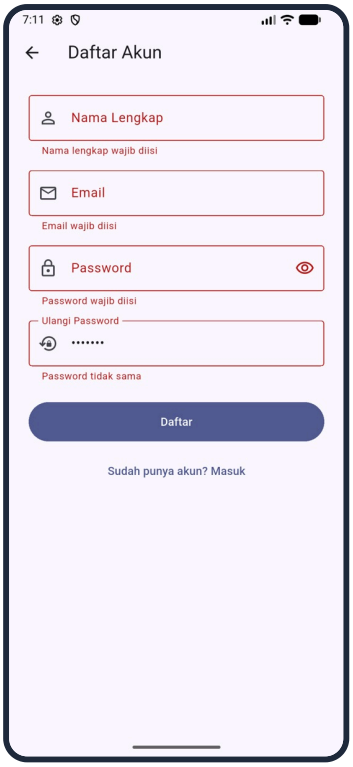
```
                                width: 22,
                                height: 22,
                                child: CircularProgressIndicator(
                                  strokeWidth: 2.5,
                                ),
                              )
                            : const Text('Daftar'),
                      ),
                      const SizedBox(height: 12),
                      TextButton(
                        onPressed: isLoading ? null : () => Get.back(),
                        child: const Text('Sudah punya akun? Masuk'),
                      ),
                    ],
                  );
                }),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

Membedah RegisterView

- `_validateConfirm` adalah validator yang membandingkan dua field. Ia dibuat sebagai method (bukan di `Validators`) karena butuh akses ke `_passwordController`.
- `textCapitalization: TextCapitalization.words` membuat keyboard otomatis menulis huruf kapital di awal setiap kata nama.
- `switch (result)` menangani tiga kasus:
  - `loggedIn` (Confirm email OFF): sesi langsung aktif dan AuthGate di *bawah* halaman ini sudah berganti ke Home. Cukup `Get.back()` untuk menutup halaman register, maka user langsung melihat Home.
  - `needsEmailConfirmation` (Confirm email ON): tampilkan dialog, lalu `Get.back()` kembali ke Login.
  - `failed`: tampilkan SnackBar berisi `errorMessage` dari controller.
- `if (!mounted) return;` diletakkan tepat setelah `await _auth.register(...)`, sebelum memakai `context`. Setelah `await showDialog` pun kita cek `mounted` lagi.
- Dialog memakai `showDialog` (Modul 05) dan menutup dirinya sendiri dengan `Navigator.of(dialogContext).pop()`, agar yang tertutup adalah dialog, bukan halaman register. Setelah dialog ditutup, `Get.back()` menutup halaman register.
- Controller diambil dengan `Get.find`, sama seperti di LoginView. Tidak ada controller baru yang dibuat: kedua halaman berbagi **satu** `AuthController` sehingga status loading dan sesi selalu sinkron.



RegisterView dibuka dari tombol "Daftar di sini" (`Get.to`)



Validasi: field kosong dan password tidak sama



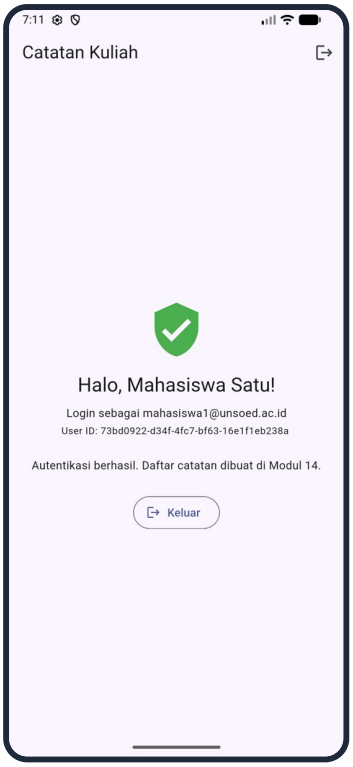
Alamat @example.com ditolak (kode `email_address_invalid`): pesan error tampil di SnackBar

Jalankan aplikasi!

Semua file Bagian 1 sudah lengkap. Jalankan:

```
flutter analyze
flutter run --dart-define-from-file=env.json
```

`flutter analyze` harus melaporkan *No issues found!*. Lalu coba register, dan jika Confirm email OFF Anda langsung masuk ke Home sementara:



Setelah login: HomeView sementara menampilkan nama dari metadata

9. Pengujian & Troubleshooting

Fitur autentikasi wajib diuji secara sistematis karena menyangkut keamanan dan menjadi pintu masuk seluruh aplikasi. Kerjakan checklist berikut dan catat hasilnya di laporan praktikum.

Checklist pengujian manual

No	Skenario	Langkah	Hasil yang diharapkan
1	Validasi form login	Tekan Masuk dengan form kosong, lalu isi email tanpa "@"	Pesan merah di bawah field, tidak ada request ke server
2	Show/hide password	Ketik password, tekan ikon mata	Teks password terlihat/tersembunyi bergantian
3	Validasi register	Isi password dan ulangi password berbeda	"Password tidak sama"
4	Register (Confirm email OFF)	Isi semua field dengan benar, tekan Daftar	Langsung masuk Home dan nama tampil. Baris baru muncul di tabel `profiles`
5	Register (Confirm email ON)	Aktifkan Confirm email di dashboard, register email lain	Dialog "Cek Email Anda", kembali ke Login. Login sebelum konfirmasi menghasilkan "Email belum dikonfirmasi"
6	Register email yang sama (Confirm OFF)	Register ulang dengan email dari skenario 4	SnackBar "Email sudah terdaftar. Silakan login."
7	Login password salah	Email benar, password salah	SnackBar "Email atau password salah."
8	Klik ganda tombol	Tekan Masuk berkali-kali dengan cepat	Tombol nonaktif + spinner, hanya satu request
9	Login tanpa internet	Matikan Wi-Fi/data emulator, coba login	SnackBar "Tidak dapat terhubung ke server..."
10	Logout	Tekan ikon logout di Home	Kembali ke Login tanpa bisa "back" ke Home
11	Sesi tersimpan	Login, tutup aplikasi sepenuhnya (swipe dari recent apps), buka lagi	Langsung tampil Home tanpa login ulang
12	Lupa env.json	Jalankan `flutter run` tanpa `--dart-define-from-file`	Layar petunjuk ConfigErrorApp

> 💡 **Mematikan internet di emulator**
>
> Tarik panel notifikasi emulator lalu matikan Wi-Fi dan Mobile data, atau aktifkan Airplane mode. Di HP fisik cukup aktifkan mode pesawat.

Troubleshooting autentikasi

Gejala	Penyebab	Solusi
Layar "SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY belum diisi"	Run tanpa `--dart-define-from-file=env.json`	Tambahkan argumen, atau atur `launch.json` (Modul 12 section 4)
Invalid API key / 401	Key salah salin atau memakai key dari project lain	Salin ulang dari Project Settings → API Keys
Database error saving new user	Fungsi trigger `handle_new_user` error	Cek Logs → Postgres. Pastikan tabel `public.profiles` ada dan nama kolom benar
Register sukses tetapi tidak bisa login	Confirm email ON dan link belum diklik	Klik link di email, atau matikan Confirm email untuk praktikum
Register gagal "Alamat email tidak valid"	Supabase menolak domain contoh seperti `@example.com` (kode `email_address_invalid`), dan server email bawaan Supabase hanya mengirim ke alamat anggota tim project	Pakai alamat email sungguhan untuk uji register, atau buat akun uji lewat **Authentication → Users → Add user** dengan **Auto Confirm User**. Untuk produksi, pasang SMTP sendiri di **Authentication → Emails → SMTP Settings**
Email konfirmasi tidak pernah datang	Batas email bawaan Supabase / masuk folder spam	Cek spam, tunggu, atau matikan Confirm email
Kolom `full_name` di profiles kosong	Key metadata berbeda (`fullName` vs `full_name`)	Samakan key di `signUp(data: ...)` dan di SQL trigger
Setelah login, layar tetap di Login	`home:` di GetMaterialApp bukan `AuthGate`, atau `currentSession` dibaca di luar builder `Obx`	Periksa `main.dart`, `auth_gate.dart`, dan `onInit` di `AuthController`
Setelah register (Confirm OFF), halaman Register tidak tertutup	Lupa `Get.back()` pada kasus `loggedIn`	Lihat `switch` di RegisterView
Error *Failed host lookup*	Tidak ada internet, emulator offline, atau URL salah ketik	Cek koneksi emulator dan isi `SUPABASE_URL`

Ringkasan Modul 13

- Arsitektur berlapis pola MVC: View → Controller (GetX) → Service/Repository → Supabase/Hive.
- Tabel `profiles` diisi otomatis oleh trigger `security definer` dari `raw_user_meta_data`.
- `AuthService` membungkus Supabase Auth, sedangkan `AuthController` menyimpan loading/error/sesi secara reaktif (`.obs`), berlangganan `onAuthStateChange` di `onInit`, dan menerjemahkan kode error.
- `AuthGate` + `Obx` yang membaca `currentSession` membuat navigasi login/logout otomatis tanpa `Get.off` atau Navigator manual.
- Login dan Register memakai Form + validator + show/hide password + tombol loading + SnackBar.

Di Modul 14 kita melanjutkan kode yang **sama**: menambah tabel `notes`, cache Hive, dan mengganti HomeView sementara dengan daftar catatan lengkap.

Tugas Praktikum: Praktikum 13: Autentikasi Aplikasi Catatan Kuliah

Sasaran Praktikum

Membangun fitur register, login, logout, dan sesi tersimpan yang aman dengan Supabase Auth, mengikuti arsitektur MVC berlapis dengan GetX.

Langkah Pengerjaan

1. Buat proyek `catatan_kuliah`, pasang package dengan `flutter pub add supabase_flutter get hive_ce hive_ce_flutter`, dan siapkan `env.json` + `.gitignore`.
2. Jalankan SQL tabel `profiles`, fungsi `handle_new_user`, trigger, dan policy RLS di SQL Editor. Simpan SQL di `supabase/setup.sql`.
3. Implementasikan `env.dart`, `validators.dart`, `auth_service.dart`, dan `controllers/auth_controller.dart` sesuai section 4–5.
4. Implementasikan `views/auth_gate.dart`, HomeView sementara, dan `main.dart` yang mendaftarkan `AuthController` dengan `Get.put(..., permanent: true)` sebelum `runApp` (section 6).
5. Implementasikan LoginView dan RegisterView lengkap dengan validasi, show/hide password, tombol loading di dalam `Obx`, dan SnackBar error (section 7–8).
6. Pastikan `flutter analyze` tidak menemukan masalah.
7. Jalankan 12 skenario pada checklist section 9. Lampirkan screenshot minimal untuk skenario 4, 5, 7, 10, dan 11.
8. Tambahan: tampilkan nama dari tabel `profiles` (bukan metadata) di HomeView memakai `select().eq('id', userId).maybeSingle()`. Simpan hasilnya di state reaktif controller dan muat di `onInit`.

Output yang Diharapkan

Aplikasi dapat register (dengan nama tersimpan di tabel profiles), login, logout, mengingat sesi setelah aplikasi ditutup, dan menampilkan pesan error berbahasa Indonesia untuk password salah, email terdaftar, email belum dikonfirmasi, dan tidak ada koneksi.

Rubrik Penilaian

Setup Supabase: tabel profiles, trigger & RLS benar (20%), Arsitektur MVC: AuthService & AuthController terpisah dari View, controller tanpa BuildContext (20%), AuthGate & navigasi otomatis (15%), Form login/register: validasi, show/hide, loading, SnackBar (25%), Laporan pengujian 12 skenario (15%), Kerapian kode & flutter analyze bersih (5%).

Kuis Pemahaman

1. Dalam arsitektur proyek ini, class mana yang boleh memanggil supabase.auth secara langsung?

A. LoginView

B. AuthController

C. AuthService

D. Semua class boleh

2. Mengapa fungsi handle_new_user diberi atribut 'security definer'?

A. Agar fungsi berjalan lebih cepat

B. Agar fungsi berjalan dengan hak pembuatnya sehingga bisa insert ke profiles walaupun RLS aktif dan user belum login

C. Agar password user dienkripsi

D. Agar trigger hanya berjalan sekali seumur hidup

3. Nilai apa yang menandakan bahwa signUp berhasil tetapi user harus mengonfirmasi email terlebih dahulu?

A. response.user == null

B. response.session == null

C. AuthException dengan code invalid_credentials

D. response.session != null

4. Setelah login sukses, bagaimana aplikasi berpindah dari LoginView ke HomeView?

A. LoginView memanggil Get.off(() ⇒ const HomeView())

B. AuthController menerima event signedIn dari onAuthStateChange dan mengisi sesi reaktif, lalu Obx di AuthGate membangun HomeView

C. AuthController memanggil runApp ulang

D. User harus menekan tombol refresh

5. Di mana AuthController berlangganan stream onAuthStateChange, dan kapan langganan itu dibatalkan?

A. Di build() AuthGate, dibatalkan saat widget di-rebuild

B. Di onInit() controller, dibatalkan di onClose()

C. Di constructor LoginView, tidak perlu dibatalkan

D. Di main(), dibatalkan saat runApp selesai

6. Apa fungsi blok 'finally' pada method signIn di AuthController?

A. Menampilkan SnackBar

B. Memastikan isLoading kembali false baik login berhasil maupun gagal

C. Mengulang login jika gagal

D. Menghapus sesi

7. Pada LoginView, mengapa onPressed tombol Masuk diisi null ketika isLoading bernilai true?

A. Agar warna tombol berubah saja

B. Untuk menonaktifkan tombol sehingga mencegah request login ganda akibat klik berulang

C. Agar keyboard tertutup

D. Karena Flutter mewajibkannya

8. Mengapa pesan error untuk kode invalid_credentials dibuat 'Email atau password salah' dan bukan 'Email tidak terdaftar'?

A. Agar pesan lebih pendek

B. Supaya penyerang tidak bisa menebak email mana yang terdaftar (user enumeration)

C. Karena Supabase tidak tahu emailnya

D. Karena Bahasa Indonesia tidak punya kata yang tepat

9. Mengapa AuthController didaftarkan dengan Get.put(..., permanent: true) di main() sebelum runApp?

A. Agar controller dibuat ulang setiap kali halaman berganti

B. Agar satu instance AuthController tersedia untuk semua halaman lewat Get.find dan tidak dihapus GetX selama aplikasi berjalan

C. Karena GetMaterialApp tidak bisa dipakai tanpa controller

D. Agar AuthController bisa menampilkan SnackBar

Kunci Jawaban & Pembahasan

1. C — AuthService adalah satu-satunya lapisan yang berbicara dengan Supabase Auth. Controller memanggil service, sedangkan View memanggil controller. Pemisahan ini memudahkan pengujian dan penggantian backend.

2. B — Saat register, request belum membawa sesi login sehingga RLS menolak insert. security definer membuat fungsi berjalan dengan hak pemiliknya (admin).

3. B — Jika Confirm email aktif, akun dibuat (user terisi) tetapi session bernilai null sampai link konfirmasi diklik.

4. B — AuthController berlangganan stream status login. Begitu ada session, nilai Rx berubah dan Obx di AuthGate otomatis mengembalikan HomeView tanpa navigasi manual.

5. B — onInit dipanggil sekali saat controller didaftarkan dengan Get.put, sehingga langganan dibuat satu kali saja. onClose adalah pasangannya untuk membatalkan StreamSubscription agar tidak bocor memori.

6. B — finally selalu dieksekusi setelah try/catch, sehingga spinner pasti berhenti dan tombol aktif kembali dalam semua kondisi.

7. B — Tombol dengan onPressed null otomatis nonaktif. Ini mencegah user mengirim beberapa request login bersamaan.

8. B — Pesan gabungan adalah praktik keamanan standar. Membedakan keduanya membocorkan informasi tentang akun yang ada.

9. B — Status login dibutuhkan di seluruh aplikasi. Get.put di main() menjalankan onInit lebih awal, dan permanent: true mencegah controller ikut terhapus saat halaman (misalnya RegisterView) ditutup.